

- 第7次作业

第7次作业

1.编写函数，存储一个文件夹的信息，要求用深度优先和广度优先两种方式

bfs

```
#include <iostream>
#include <filesystem>
#include <string>
#include <unordered_map>
#include <vector>
#include <variant>
namespace fs = std::filesystem;
using namespace std;
using var = std::variant<string, int, bool>;
vector<unordered_map<string, var>> dirList;
int id = 0;

void bfs(std::string path, int parent_id) {
    if (fs::is_regular_file(path)) {
        return;
    }

    try {
        for (auto& entry: std::filesystem::directory_iterator{path}) {
            dirList.push_back({
                {"id", id++},
                {"parent_id", parent_id},
                {"path", entry.path().string()},
                {"name", entry.path().filename().string()},
                {"is_dir", entry.is_directory()}
            });
        }
    } catch (const fs::filesystem_error& ex) {
        cerr << "无法访问目录: " << path << ", 错误: " << ex.what() << endl;
    }
}

int main() {
    std::string path = "D:/CLion 2026.1.1";
    dirList.push_back({
        {"id", id++},
        {"parent_id", -1},
        {"path", path},
        {"name", "data structure"},
        {"is_dir", true}
    });
}
```

```

for (int i = 0; i < dirList.size(); i++) {
    unordered_map<string, var> dir = dirList[i];
    if (get<bool>(dir["is_dir"])) {
        bfs(get<string>(dir["path"]), get<int>(dir["id"]));
    }
}

cout << "总共找到 " << dirList.size() << " 个项目" << endl;

return 0;
}

```

dfs

```

#include <iostream>
#include <vector>
#include <filesystem>
#include <variant>
#include <unordered_map>

namespace fs = std::filesystem;
using namespace std;
using var = variant<int, string, bool>;
vector<unordered_map<string, var>> vect;
int id=0;
void dfs(string dir, int parent_id){
    if (dir.empty() || fs::is_regular_file(dir)) return;
    for (auto &item : fs::directory_iterator(dir)){
        unordered_map<string, var> m;
        m["id"]=id++;
        m["name"]=item.path().filename().string();
        m["is_file"]=fs::is_regular_file(item.path());
        m["parent_id"]=parent_id;
        m["path"]=item.path().string();
        vect.push_back(m);
        if (fs::is_directory(item.path())){
            continue;
        }
        dfs(item.path().string(), m["id"].index());
    }
}
int main ()
{
    unordered_map<string, var> m;
    string path="D:/CLion 2026.1.1";
    m["id"]=id;
    m["name"]=fs::path(path).filename().string();
    m["is_dir"]=true;
    m["path"]=path;
    m["parent_id"]=-1;
    vect.push_back(m);
    dfs(path, 0);
    return 0;
}

```

```
}
```

2.在深度优先中，一般采用递归的方式。根据第一题的代码，画一个树型的文件夹结构（就像我在课堂上画的那样，——任意的），然后写出每次递归调用的函数参数（实参）和返回位置。

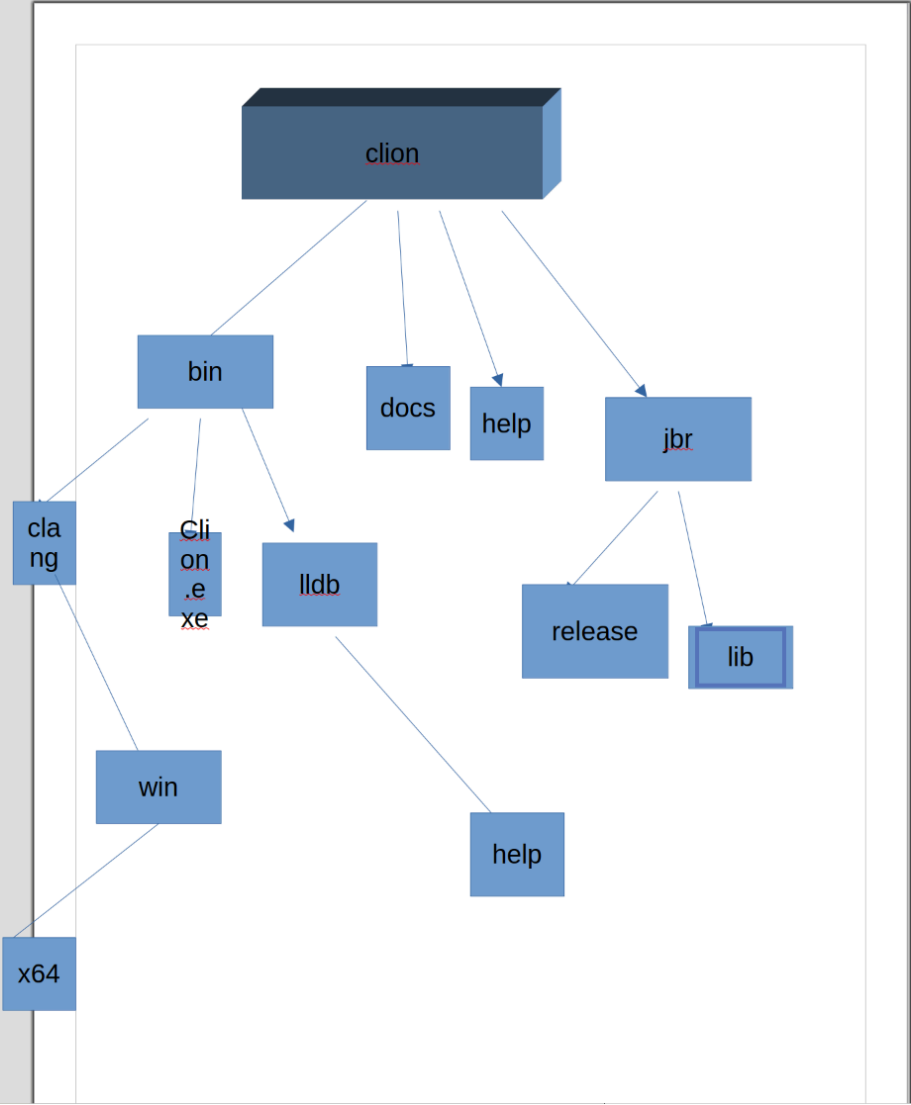
```
//设有递归调用
#include <iostream>
using namespace std;
int add(int n) {
    if (n == 0) {
        return 0;    // 返回位置 0
    }
    return n + add( n-1 );    // 返回位置1
} // 若程序自由结束，则此处也有一个返回位置 3
int main() {
    cout<<add(3)<<endl; // 5050
}
```

第几次调用	参数	返回位置
1	3	1
2	2	1
3	1	1
4	0	0

你要做的：

先用PPT画一张文件夹树，深度超过4，总节点数（包含叶子数）超过10个

【图】



【表】

序号 节点名称 类型 深度 父节点

1 clion 目录 0 -1

2 bin 目录 1 1

3 clang 目录 2 2

4 win 目录 3 3

5 x64 目录 4 4

6 CLion.exe 文件 2 2

7 lldb 目录 2 2

8 help 目录 3 7

9 docs 文件 1 1

10 help 文件 1 1

11 jbr 目录 1 1

12 release 目录 2 11

13 lib 文件 2 11